基础问答

1. **一张RGB三通道彩色图片是如何被处理成可用于训练的数据？张量又是什么？**

   - 处理流程：

   1. 读取图片后把像素值从0-255的整数归一化到0-1的浮点数范围，消除数值量级差异对训练的影响
   2. 统一缩放裁剪到固定尺寸，比如224×224，保证所有输入样本维度完全一致
   3. 按照PyTorch的张量格式调整维度顺序，从常规的(H, W, 3)转换成(3, H, W)，把通道维度放到最前面
   4. 用数据集的均值和方差对三个通道分别做归一化

      - 张量：

      深度学习框架里的多维数组：标量是0维张量，向量是1维张量，图片就是3维张量（通道、高、宽），批量图片组成的训练批次就是4维张量（批次大小、通道、高、宽）

2. **一个卷积神经网络分为哪几层？**

   核心分为5类功能层：

   1. 输入层：接收预处理完成的图片张量，定义输入的尺寸、通道数
   2. 卷积层：用卷积核提取图片的边特征，是CNN的核心计算层
   3. 池化层：下采样压缩特征图尺寸，降低计算量，同时保留核心特征
   4. 激活层：给网络引入非线性能力，让模型可以拟合复杂的分类边界
   5. 全连接层：把前面提取到的分布式特征映射到最终的分类类别维度，输出分类结果

3. **什么是卷积，数学严格意义上的卷积和神经网络中的卷积操作有区别吗？卷积核、步长是什么？**

   - 有区别：数学上严格意义上的卷积需要把卷积核旋转180度再和对应位置的元素相乘求和；而神经网络中的卷积不需要旋转卷积核，直接就把对应的元素相乘求和就好了，这样就可以减少无意义的一步操作，提高计算的效率。

   - 卷积核:是一个可学习权重的小矩阵，用来在图片上滑动提取局部特征

   - 步长：卷积核每次在特征图上滑动移动的像素步数，步长大于1的时候可以直接实现下采样，缩小特征图尺寸

4. **池化有哪些类型，其作用分别有什么？感受野是什么？**

   - 池化类型和作用：

1. 最大池化：取池化窗口内的最大值，能最强保留边缘、纹理这类显著特征
2. 平均池化：取池化窗口内所有元素的平均值，更多保留整体背景信息
3. 全局平均池化：直接对整个特征图求平均值，把整个二维特征图压缩成一个一维向量，替代传统全连接层，大幅减少参数量
   - 感受野：卷积神经网络里某一层输出的单个特征点，在原始输入图片上能覆盖到的像素区域大小。随着卷积核逐层提取特征，感受野也不断变大，所以越深层的感受野的区域越大。

5. **激活层在此的作用？最后卷积神经网络会输出什么？**
   - 作用：给网络引入非线性函数，使卷积神经网络具有非线性的能力，
   - 输出：该项目中对应不同类别的概率分布向量，经过softmax处理后，每个位置的数值代表当前图片属于该类别的置信度，最大的那个数对应的类别就是模型预测的结果。

6. **什么是超参数？如何设定？**
   - 超参数：超参数是在模型训练开始之前，需要人工预先设定、训练过程中不会被模型自动更新的参数，比如学习率、批次大小、网络层数、卷积核大小这些都属于超参数。
   - 如何设定:
     1. 参考同类型公开任务的成熟基线参数
     2. 训练过程中根据损失下降趋势动态调整

7. **什么是优化器，有哪些类型？**
   - 优化器:用来更新神经网络中可训练权重的算法，核心目标是最小化损失函数，让模型的预测结果越来越贴近真实标签。
   - 类型：
     1. SGD随机梯度下降：最基础的优化器，配合动量使用可以稳定收敛，泛化能力很强
     2. Adam：自适应学习率优化器，自动给不同参数调整学习率，收敛速度快，对初始学习率不敏感，是现在图像分类任务里最常用的优化器

8. **DataLoader 与 DataSet是什么及其使用？（可调库）**
   - Dataset是PyTorch里的抽象数据集类，需要继承它重写__len__和__getitem__两个方法，用来定义花卉数据集的读取规则、预处理逻辑，负责单张样本的加载。
   - DataLoader是批量数据加载器，它基于你写好的Dataset，自动完成批量打包、多线程并行加载、打乱样本顺序的工作，直接给训练循环源源不断输出批量的训练数据，不用你自己手动写循环切分数据集。
   - 使用的时候先自定义FlowerDataset继承DataSet写好读取逻辑，然后把它传入

DataLoader，设置batch_size、shuffle=True、num_workers参数，后续训练的时候直接for循环遍历DataLoader就能拿到每一批训练数据。

V-模型的优化与微调

---

1. **我如果一直堆叠网络的层数，岂不是能无限的提高模型的能力？模型层数越多、结构越深，性能就一定越好吗？**

   一直堆叠网络层数并不能无限的提高模型的能力。模型层数越高，越有可能带来一些负面影响：

   1.梯度消失：当网络深度超过一定阈值之后，反向传播的梯度会在层层传递中逐渐消失，深层网络的参数几乎得不到有效更新

   2.容易过拟合：层数越多，参数越多，模型越复杂，就可能把所有训练集的数据都记住，造成过拟合。

   3.计算量更大：层数越多，每一次运行时越往深处的参数越多，计算量越大。

2. **学习现代新的网络结构(如ResNet、VGG)，并尝试利用其训练模型，对比两个模型的能力。**

   - VGG：是经典的纯堆叠3×3卷积的直连网络，结构非常规整容易实现，但是层数堆到16、19层之后很容易出现梯度退化，训练收敛速度偏慢，在花卉数据集上最终测试准确率大概在70%左右。

   - ResNet：引入了残差跳跃连接（前向传播有两条并行通道，反向传播可以直接走跳跃通道），直接解决了深层网络的梯度消失问题，哪怕堆到几十层也能稳定收敛，特征提取能力更强，在同一个花卉数据集上最终测试准确率能做到80%以上，泛化能力和训练效率都明显优于同深度的VGG。

3. **模型训练好了该如何使用呢？难道每次使用前都需要重新训练一次？请学习模型的保存与载入方法，将自己的模型保存在本地，自己在网上找到图片后丢给模型分类。**

   - 保存模型：训练完成后直接用torch.save(model.state_dict(), "flower_model.pth")把训练好的模型权重（一轮一轮训练过程中不断修改的参数的合集）持久化到本地文件里，不用保存整个网络结构，文件体积小也更灵活。

   - 加载模型：使用的时候先把你定义好的网络结构代码写好（模型结构不管在哪里都是固定可套用的），再用model.load_state_dict(torch.load("flower_model.pth"))把本地的权重加载进网络，直接就能获得训练完成的模型效果。

   - 推理使用：加载完权重之后把模型设为eval评估模式（会自动关闭训练阶段专属的特殊逻辑以及一些梯度计算），把网上找的花卉图片做和训练集完全一致的预处理，输入模型就能直接输出分类结果